

Exercício Prático

Objetivo

Você recebeu um repositório com uma arquitetura rodando localmente (sem Docker e sem Nginx):

- gateway (porta 8080) → faz proxy para a API
- api (porta 3000) → recebe eventos e expõe stats
- worker (porta 3001) → processa eventos e grava agregados em disco
- data/ → simula volume (persistência local)

Sua missão

Containerizar os serviços com Docker, criar um **docker-compose** com **volume compartilhado** e colocar um **Nginx na frente** como gateway/reverse proxy.

Entregáveis

Crie os arquivos abaixo:

1) Dockerfiles (3 arquivos)

- api/Dockerfile
- worker/Dockerfile
- gateway/Dockerfile (*opcional, dependendo do modo escolhido com Nginx — ver abaixo*)

2) docker-compose

- docker-compose.yml na raiz do projeto

3) Nginx reverse proxy

- nginx/nginx.conf

Arquitetura alvo

Portas (externas e internas)

- **Nginx:** exposto em `http://localhost:8080`
- **API:** interna na rede Docker :3000
- **Worker:** interno :3001
- **Gateway Java:**
 - pode ser removido (recomendado) OU manter rodando interno (extra)

Rotas no Nginx

- GET /health → responde "ok" (health check)
- POST /api/events → encaminha para API
- GET /api/stats → encaminha para API

Persistência (volume)

- Os containers devem compartilhar um volume/pasta para:
 - data/inbox/events.ndjson
 - data/inbox/offset.txt
 - data/out/stats.json

Regras do exercício

Regra 1 — API não processa eventos

A API **apenas grava eventos** em `data/inbox/events.ndjson`.

Regra 2 — Worker processa em background

O worker roda em loop/scheduler, consome eventos novos e atualiza `stats.json`.

Regra 3 — Cliente não conhece API/Worker

O cliente deve falar **apenas com Nginx na porta 8080**.

Checklist (passo a passo do que precisa estar pronto)

A) Dockerfiles

- Criar `api/Dockerfile` que gera uma imagem executável
- Criar `worker/Dockerfile` que gera uma imagem executável
- (Opcional) Criar `gateway/Dockerfile` caso você mantenha o gateway Java no compose

Recomendação: use multi-stage build (Maven build → JRE runtime).

B) docker-compose.yml

- Subir pelo menos 3 serviços: `nginx`, `api`, `worker`
- Configurar rede interna (default já resolve)
- Mapear `nginx:8080 -> host:8080`
- Garantir que `api` e `worker` compartilham o mesmo volume `data`
- `depends_on` para garantir ordem mínima de inicialização

C) Nginx reverse proxy

- Criar `nginx/nginx.conf`
- Criar upstream para API
- Roteamento por path:
 - `/api` → API
 - `/health` → health
- Adicionar headers de proxy
- Configurar timeouts
- Habilitar logs de acesso e erro

Critérios de Aceite

Critério 1 — Funciona via Nginx na porta 8080

- ✓ O sistema deve funcionar com o cliente chamando **somente** `localhost:8080`.

Critério 2 — Persistência real via volume

- ✓ Se você derrubar os containers e subir de novo, os dados devem continuar:

- `events.ndjson`
- `stats.json`
- `offset.txt`

Critério 3 — Logs e proxy corretos

- ✓ O Nginx deve logar as chamadas e encaminhar corretamente.

Critério 4 — Rotas separadas

- ✓ `/api/*` e `/health` devem existir e funcionar.

Critério 5 — Worker realmente processa

- ✓ Stats só mudam depois que o worker roda.

Dicas

Dica 1 — Caminho do volume

Dentro do container, use um caminho padrão, por exemplo:

- `/data/inbox/events.ndjson`
- `/data/out/stats.json`

Você provavelmente vai precisar ajustar o `application.yml` para apontar para `/data`.

Exemplo:

```
app:
  dataDir: /data
```

Dica 2 — Compose com volume nomeado

Exemplo de volume:

```
volumes:
  app_data:
```

E montar nos serviços:

```
volumes:
  - app_data:/data
```

Dica 3 — Nginx enxergar API pelo nome do serviço

No compose, se o serviço chama `api`, o Nginx usa:

- `http://api:3000`

Dica 4 — Headers úteis no proxy

No `nginx.conf`, use:

- `proxy_set_header Host $host;`
- `proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;`

Dica 5 — Timeouts

Para evitar travar em caso de erro:

- `proxy_connect_timeout`
- `proxy_read_timeout`
- `proxy_send_timeout`

Comandos de teste

Subir tudo

```
docker compose up --build
```

Health

```
curl http://localhost:8080/health
```

Enviar eventos

```
curl -X POST http://localhost:8080/api/events \
  -H "Content-Type: application/json" \
  -d '{"type":"click","source":"web"}'
curl -X POST http://localhost:8080/api/events \
  -H "Content-Type: application/json" \
  -d '{"type":"signup","source":"mobile"}'
```

Stats

```
curl http://localhost:8080/api/stats
```

Extra

Se quiser dar “nível 2” para alunos mais rápidos:

- ✓ Criar rota `/worker/health` no Nginx apontando para worker
- ✓ Adicionar `limit_req` no Nginx (rate limit)
- ✓ Adicionar `gzip on;`
- ✓ Criar um Makefile com comandos `make up`, `make down`, `make logs`